

Double-UV Relative-Momentum Enhancement in the GL00 $\gamma\gamma \rightarrow \gamma\gamma$ Integrand

Investigation note

June 15, 2026

Abstract

This note documents the current understanding of a large-weight feature seen in the generated GL00 two-loop integrand for $\gamma\gamma \rightarrow \gamma\gamma$ with the nested integrated UV counterterm included. The issue is not an ordinary uncanceled UV divergence in a generic double-UV direction. The problematic samples lie close to a relative-momentum ridge: two sampled loop-momentum-basis (LMB) rays become hard, nearly collinear, and have comparable radii, so that one physical internal momentum is a small difference of the two hard basis momenta. The resulting enhancement is locally integrable, but it creates large Monte-Carlo variance because the current sampling channels do not parameterize that relative momentum directly. LMB multichanneling is not merely a random choice of LMB: it includes a normalized OSE-product prefactor, and that prefactor strongly suppresses inappropriate channels near the edge-small limit. The remaining question is how much this discrete reweighting, and in particular its exponent α , can reduce the variance without an additional continuous relative-momentum map. A fixed-ray option was added to the UV profiler to make this non-generic direction visible in profiling runs.

1 Executive summary

For the regenerated all-LMB setup, GAMMALOOP reports all eleven loop momentum bases as active channels:

$$(5, 10), (4, 10), (5, 9), (4, 9), (5, 8), (4, 8), (5, 7), (4, 7), (5, 6), (4, 6), (4, 5).$$

This required one small display/introspection correction: the display command must read the generated multichannel setup instead of recomputing the old heuristic.

The short all-LMB Monte-Carlo run still found its largest signed evaluations in channels (5,10) and (5,9), not in a channel that explicitly uses edge 4 as a basis variable. That is consistent with a variance problem rather than a pure channel-enumeration problem. In the original graph-generation basis,

$$e_4 = K_0, \tag{1}$$

$$e_5 = -K_0 + K_1 + P_0, \tag{2}$$

$$e_9 = K_1, \tag{3}$$

$$e_{10} = -P_2 + K_1 + P_0 + P_1. \tag{4}$$

In channel (5,10), where $u = e_5$ and $v = e_{10}$ are sampled directly,

$$e_4 = v - u + P_2 - P_1. \tag{5}$$

Thus a sample with u and v both hard, nearly collinear, and of similar radius maps to a much smaller edge-4 momentum. This is precisely the geometric configuration that the OSE channel weights can identify only indirectly: they can favor LMBs containing edge 4, but they do not replace the continuous variables (u, v) by common and relative momenta.

2 What the inspected weight contains

The approach plots below use the value printed by GAMMALOOP as “The evaluation of integrand”. This is the event weight passed to the integrator. The same inspect output also prints the parameterization jacobian separately and a much smaller “integrand result” before multiplication by that jacobian. Therefore the plotted quantity already includes the spherical parameterization jacobians and the conformal-map jacobians. The large-radius slope in the plots is a statement about the actual weight seen by the integrator, not merely about the unweighted analytic integrand.

3 Minimal toy model

The minimal structure needed to reproduce the observed mechanism is not a full two-loop numerator. It is enough to keep:

- two hard sampled LMB momenta u and v ;
- a physical internal momentum $q = v - u + Q$ that is a difference of those hard momenta plus a fixed external shift Q ;
- the post-subtraction and parameterization effect that leaves a positive power of the common hard radius in the event weight;
- an energy denominator sensitive to the small relative momentum.

The toy event weight used for the figures is

$$W_{\text{toy}}(u, v) = C(\hat{u}, \hat{v}, \rho) \frac{R^2}{E_q}, \quad E_q = \sqrt{|v - u + Q|^2 + q_0^2}, \quad R = \frac{|u| + |v|}{2}, \quad (6)$$

where $\rho = |v|/|u|$ and C is a smooth bounded angular factor. The constant q_0 regulates the exactly coincident toy edge and stands in for the fact that the full GL00 expression has masses, external shifts, and numerator cancellations. The power R^2 should not be read as a universal UV degree. It is the minimal effective power required to match the observed weighted large-radius approach after all jacobians are included.

Let the relative separation scale as

$$|v - u + Q| \simeq \epsilon R \quad (7)$$

for a fixed but small opening/radius mismatch ϵ . Then

$$W_{\text{toy}} \simeq \frac{R^2}{\epsilon R} = \frac{R}{\epsilon}. \quad (8)$$

This gives a slope-one growth along a ray where the two UV momenta are scaled together while their relative mismatch is held fixed. If the two hard momenta coalesce more quickly than $1/R$, the toy model crosses over toward R^2/q_0 . For generic separated directions the coefficient is much smaller; the bad region is the ridge where $\rho \simeq 1$ and $\theta(u, v) \simeq 0$.

The singularity is locally integrable in the relative momentum. In relative variables $d^3q = |q|^2 d|q| d\Omega_q$, a $1/|q|$ factor gives

$$d^3q \frac{1}{|q|} \sim |q| d|q| d\Omega_q,$$

so the local integral is finite. Monte Carlo variance can nevertheless be large, especially when the sampling variables are the hard momenta u and v rather than the common momentum and

the relative momentum q . Figure 1 shows the corresponding radial scaling, and Figure 2 shows how the enhancement localizes near equal radius and small opening angle.

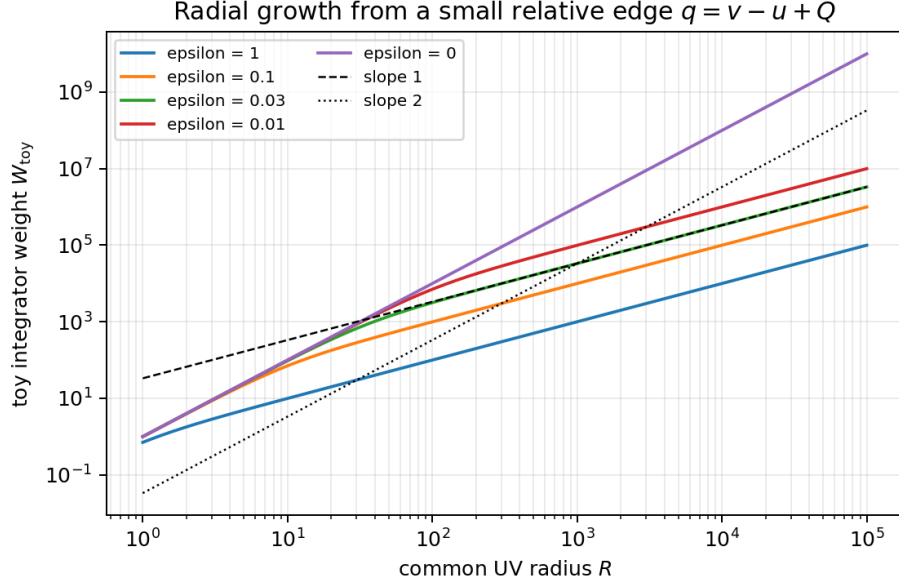


Figure 1: Toy-model radial scaling. For fixed nonzero relative mismatch ϵ , the weighted integrand approaches a slope-one growth, $W_{\text{toy}} \sim R/\epsilon$. Exact coalescence crosses over to slope two in this deliberately simplified model.

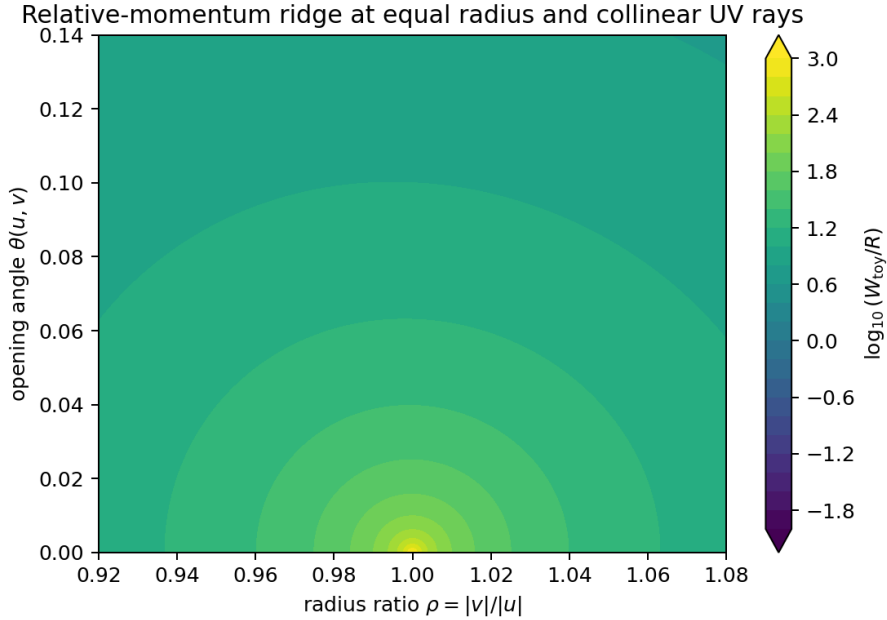


Figure 2: Toy relative-momentum ridge at fixed large common radius. The enhancement is concentrated near equal radius, $\rho = 1$, and small opening angle. Existing LMB channels can reduce coefficients, but they do not create a direct sampling variable for this ridge.

4 Standalone proxy integrand experiment

I also built a standalone Python mock-up that is closer to the actual subtracted-integrand mechanism than the purely phenomenological toy model above: `DoubleUVIntegrableSingularity/mock_gl00_integrand_proxy.py`. It defines an original relative-edge factor and the corresponding local double-UV counterterm obtained from the 3D expansion that drops the fixed external shift in the relative edge:

$$W_{\text{orig}}^{(K)}(u, v) = N_K(u, v) \frac{[(E_u + E_v)/2]^3}{E_u E_v} \frac{1}{E(v - u + Q, m_{\text{phys}})}, \quad (9)$$

$$W_{\text{UV}}^{(K)}(u, v) = N_K(u, v) \frac{[(E_u + E_v)/2]^3}{E_u E_v} \frac{1}{E(v - u, m_{\text{UV}})}, \quad (10)$$

$$W_{\text{sub}}^{(K)}(u, v) = W_{\text{orig}}^{(K)}(u, v) - W_{\text{UV}}^{(K)}(u, v). \quad (11)$$

Here u and v are the two hard sampled LMB momenta and Q is the fixed external shift in the physical relative edge. The optional integrated-CT switch only changes the smooth numerator coefficient N_K : with the switch off it contains the original numerator proxy, while with the switch on it adds a smooth integrated-self-energy proxy. It does not introduce a new denominator.

The same script also contains a second mode that mimics doing the local UV Taylor expansion in four dimensions first and only then taking the CFF expression of the resulting massive vacuum topology. The relative factor in that mode is

$$F_{\text{vac}}(q; R, m) = \frac{1}{E(q, m) [2R + E(q, m)]},$$

with the UV counterterm using a finite vacuum mass $m_{\text{UV}} = 9.188$. This is not intended to be a full CFF formula, but it keeps the two features relevant to the present question: the relative propagator is a vacuum object before CFF, and after CFF it is represented by a massive on-shell energy.

The fitted large-radius slopes of $|W_{\text{sub}}^{(K)}|$ are:

mode	ray choice	without integrated CT	with integrated CT
3D expansion	locked equal-norm collinear rays	+1.000	+1.000
3D expansion	parallel rays with 4% radius mismatch	-1.000	-1.000
3D expansion	equal radius with 1° opening angle	-1.000	-1.000
3D expansion	generic directions	-1.000	-1.000
4D first, then CFF	locked equal-norm collinear rays	+1.000	+1.000
4D first, then CFF	parallel rays with 4% radius mismatch	-1.002	-1.002
4D first, then CFF	equal radius with 1° opening angle	-0.997	-0.997
4D first, then CFF	generic directions	-1.000	-1.000

This reproduces the qualitative GL00 behavior: the subtracted proxy is UV safe for generic double-UV rays and for fixed nonzero angular/radial mismatch, but it grows when the two hard rays are locked so that $v - u$ remains small while the common radius grows. The same behavior is present with the integrated-CT proxy disabled, so the mechanism is not created by the integrated counterterm; the integrated counterterm only changes the smooth coefficient multiplying the same relative-edge structure. Importantly, the locked-ray growth is still present in the 4D-first proxy. This means the non-uniform double-UV region is not purely an artifact of differentiating the already-3D on-shell energy $E(v - u + Q)$. The radial proxy scan is shown in Figure 3.

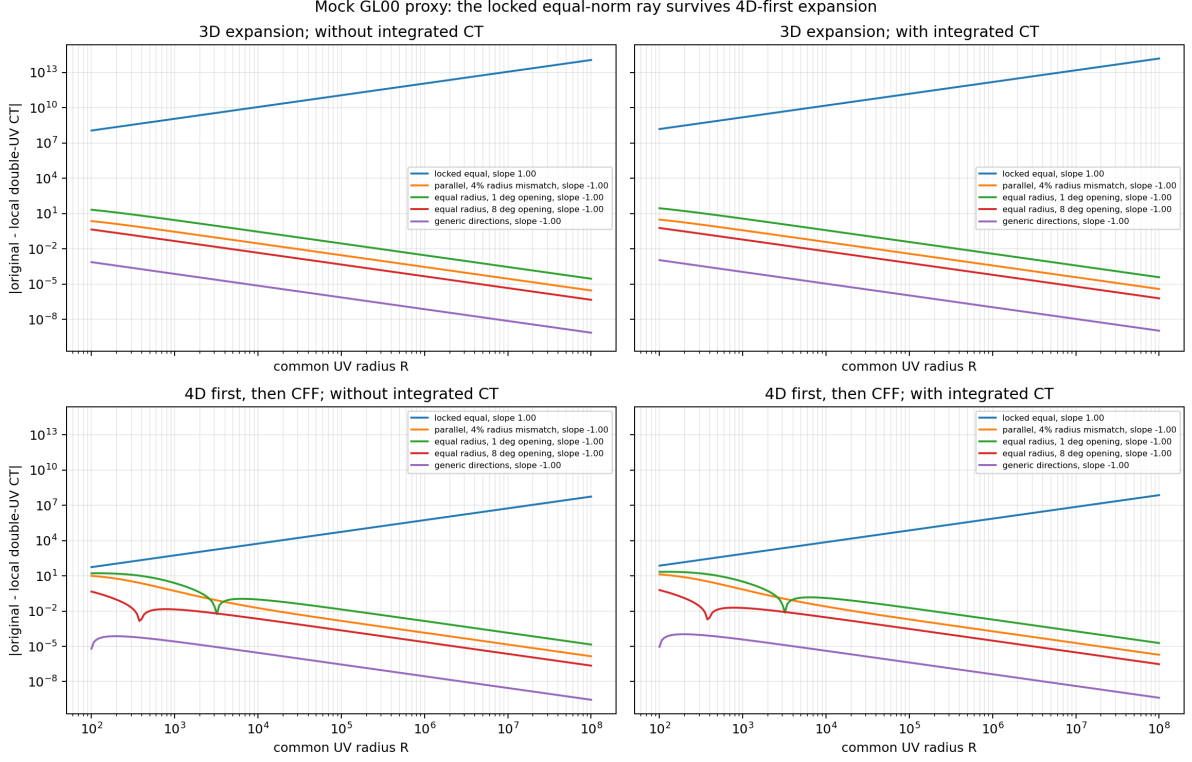


Figure 3: Standalone proxy radial scan. In both the direct 3D-expansion mode and the 4D-first vacuum-CFF mode, only exactly locked equal-norm rays grow with the common UV radius. A fixed opening angle or a fixed radius mismatch restores the expected decreasing asymptotic behavior.

At fixed hard common radius, the direct 3D proxy gives

$$|W_{\text{sub}}^{(K)}| \sim \frac{1}{|v - u|}$$

over the relative-momentum range above the small UV regulator. Multiplication by the three-dimensional relative measure gives $|v - u|^2 |W_{\text{sub}}^{(K)}| \sim |v - u|$, so the ridge is locally integrable even though it is large enough to create Monte-Carlo variance. The 4D-first proxy differs here: because the vacuum relative propagator carries a finite UV mass before CFF, the fixed-radius relative scan is flat as $v - u \rightarrow 0$ rather than $1/|v - u|$. Thus doing the derivation in 4D first can remove the literal small-relative-momentum pole introduced by a direct 3D on-shell-energy expansion. It does not, however, remove the radial non-uniformity of the double-UV Taylor expansion on the subspace where the relative four-momentum is not hard. Figure 4 shows this fixed-radius relative scan.

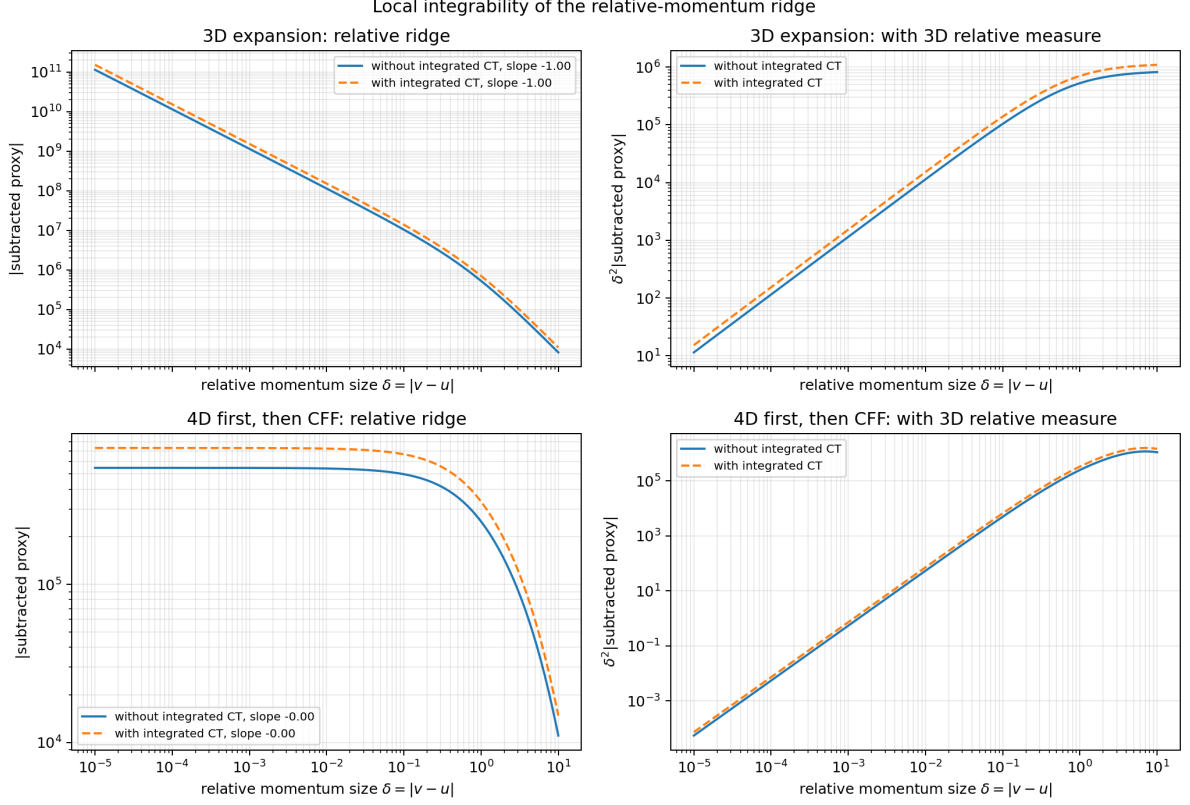


Figure 4: Standalone proxy relative-momentum scan at fixed common hard radius. The direct 3D expansion has a $1/|v - u|$ ridge. The 4D-first vacuum-CFF proxy is regularized by the massive vacuum propagator, but the three-dimensional relative measure still makes both cases locally integrable.

4.1 Removing the ridge by parameterization

The same proxy script now performs an explicit Monte-Carlo integration over a finite local domain,

$$I = \int d \log R \int_{|q| < q_{\max}} d^3 q W_{\text{sub}}^{(K)}(P = R\hat{n}, q), \quad q = v - u,$$

with $q_{\max} = 0.2$ and 2.5×10^5 samples per channel. The naive channel samples q uniformly in the three-dimensional ball. The singularity-adapted relative channel samples the radial variable with

$$p_r(r) = \frac{2r}{q_{\max}^2}, \quad p_{\text{ridge}}(q) = \frac{1}{2\pi q_{\max}^2 |q|}.$$

Because the direct 3D proxy has $W_{\text{sub}} \sim 1/|q|$, the estimator $W_{\text{sub}}/p_{\text{ridge}}$ is finite as $q \rightarrow 0$. I also tested a conservative balance-heuristic multichannel density,

$$p_{\text{multi}}(q) = 0.25 p_{\text{uniform}}(q) + 0.75 p_{\text{ridge}}(q),$$

which keeps a regular uniform component for the non-singular part of the domain.

This finite-domain experiment should be interpreted carefully. A $1/|q|$ ridge in three relative dimensions is locally integrable, and its local second moment under a flat finite ball is also finite because $\int q^2 dq |1/q|^2$ is finite at $q = 0$. The real integration problem is not just that local endpoint. It is the combination of this thin relative ridge with the hard common radius, the conformal-map Jacobian, and finite statistics. In the unbounded problem the second moment can still be effectively or genuinely divergent if the radial tail is not sampled with a compatible

density. The variance reductions below therefore demonstrate that the relative-momentum channel removes the local rare-event ridge; they are not a proof that the full UV-tail variance is finite.

integrated CT	channel	estimate	relative error	variance reduction	max $ w $
off	uniform	-2.4975×10^5	0.355%	1.0	2.07×10^8
off	ridge	-2.4836×10^5	0.012%	883.9	2.89×10^5
off	multichannel	-2.4840×10^5	0.030%	137.7	3.84×10^5
on	uniform	-3.3226×10^5	0.136%	1.0	2.39×10^7
on	ridge	-3.3177×10^5	0.012%	129.8	3.86×10^5
on	multichannel	-3.3175×10^5	0.030%	20.3	5.15×10^5

This is the desired behavior: the relative-momentum channel removes the integrable singularity from the estimator and collapses the rare-event tail. For example, without the integrated-CT proxy, the maximum absolute sampled weight drops from 2.07×10^8 to 2.89×10^5 , and the variance drops by a factor 884. The conservative multichannel mixture still reduces the variance by a factor 138. The same conclusion holds with the integrated-CT proxy enabled, although the smooth numerator changes the relative coefficient and therefore the exact reduction factor. The variance and max-weight reduction are shown in Figure 5.

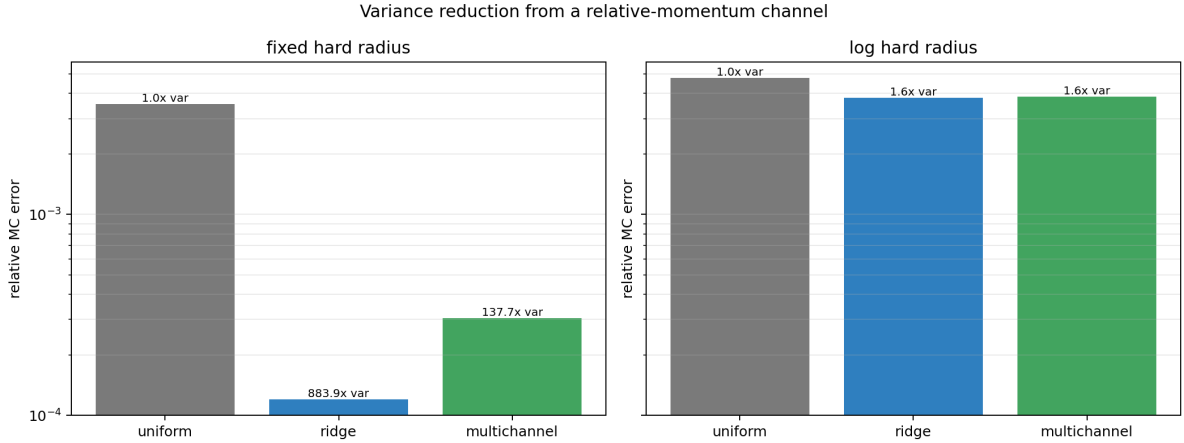


Figure 5: Monte-Carlo variance reduction in the standalone proxy. The fixed-hard-radius panel isolates the relative-momentum singularity and shows the large gain from a ridge-adapted channel. The log-hard-radius panel keeps the common-radius growth; the relative channel still helps, but the remaining variance is then dominated by the radial UV tail and would require a complementary hard-radius importance map.

When the same experiment is run in the 4D-first vacuum-CFF proxy, the ridge-adapted channel does not help at fixed hard radius: the relative scan is already flat as $q \rightarrow 0$, so oversampling small q only wastes samples. This is a useful cross-check. The relative channel should be attached to the direct 3D-CFF local counterterm structure where the $1/|k-l|$ ridge is actually present, and combined with ordinary channels through a balance heuristic.

5 GL00 approach data

The GL00 radial scan starts from the high-weight point

$$x_A = (0.9938195811740000, 0.6303276140313304, 0.6037887485521857, 0.9939524818481166, 0.6277285639451646, 0.6151209153931045).$$

At $\lambda = 1$, the two radial variables correspond to approximately

$$r_0 \simeq 4.8240 \times 10^4, \quad r_1 \simeq 4.9307 \times 10^4.$$

The two radii are therefore both in the UV and within about two percent of each other. The angular coordinates are also close; the fitted opening angle in the angular scan is

$$\theta_A \simeq 2.82 \times 10^{-2}.$$

The large-radius fit of the GAMMALOOP inspected weight gives:

case	fit range	slope in $\log W $ vs. $\log \lambda$	points
LMB multichannel off	$\lambda \geq 1$	1.00005	27
LMB multichannel on	$\lambda \geq 1$	1.00424	27

The coefficient changes strongly when LMB multichanneling is enabled, but the large-radius power does not. This is the important observation: the current multichanneling can reduce the normalization of the problematic ray, but it does not alter the radial power seen by the integrator. Figures 6 and 7 show the full and compact views of this radial scan.

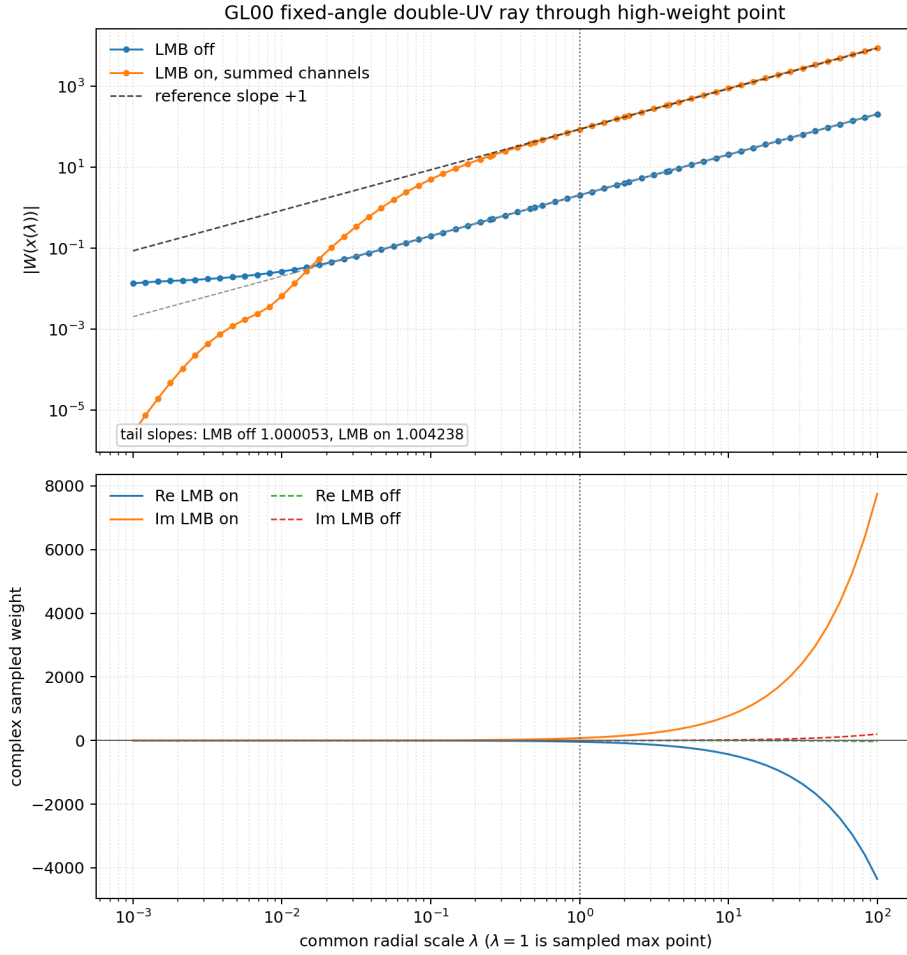


Figure 6: GAMMALOOP inspected event weight along the high-weight double-UV ray. All parameterization jacobians are included. The large-radius behavior is approximately linear in the common UV scale.

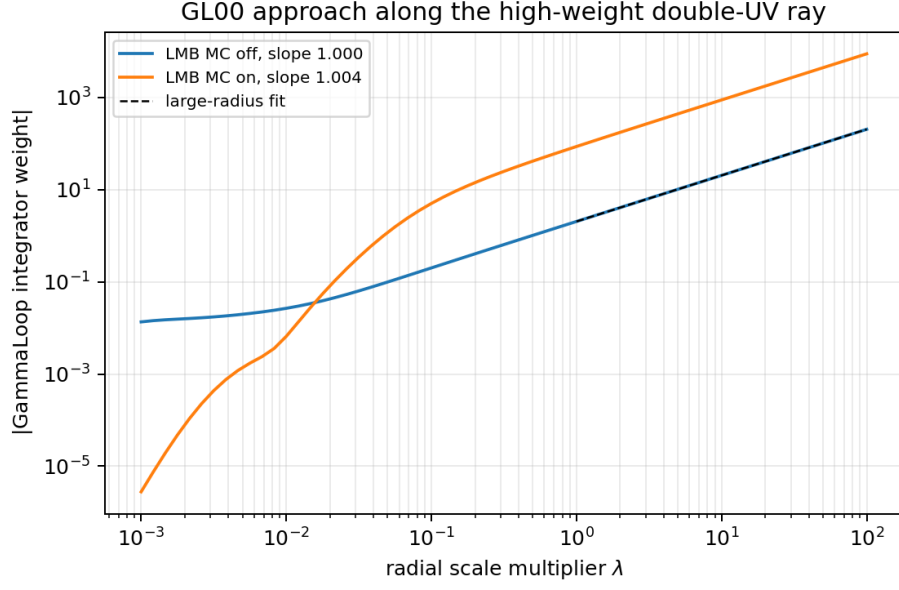


Figure 7: Compact replot of the same radial data, with large-radius fit slopes. The fit is not intended to model the low-radius region.

The angular dependence around the same point is shown in Figure 8.

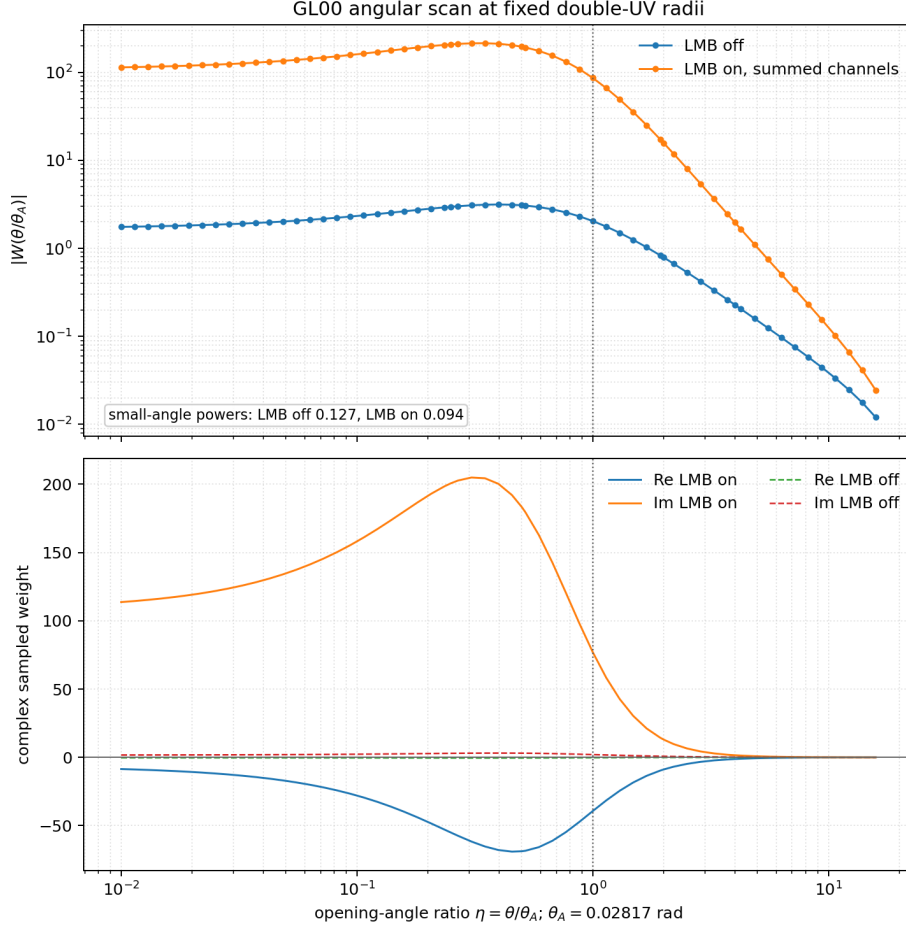


Figure 8: Angular approach scan around the same GL00 point. The coefficient is highly sensitive to the relative direction of the two hard rays, supporting the interpretation as a relative-momentum ridge rather than a generic double-UV failure.

6 Why the standard UV profile misses the enhancement

The ordinary `profile ultra-violet` command samples one random three-momentum per LMB loop variable and then rescales the selected subset. It therefore measures the generic large-momentum degree of divergence for that sampled ray. The relative-momentum ridge discussed above is a correlated configuration: two UV rays must be nearly parallel and have nearly equal radii, so that a physical edge momentum such as $v - u + Q$ stays anomalously small compared to the two hard sampled basis momenta. A random UV profile ray hits this condition with probability zero.

There is also a conceptual difference between the UV profiler and the integration-weight plots. The profiler works directly in momentum space and multiplies by the UV scaling measure factor used for the degree-of-divergence test. It does not include the full x -space parameterization jacobian, adaptive discrete-channel probability, or the conformal-map density that define the event weight seen by the integrator. This is why the standard UV profile can be clean while the integrator still sees a high-variance, locally integrable ridge.

To make the correlated direction testable, the profiler now accepts fixed UV ray data:

```
profile ultra-violet ... \
  --uv-ray-directions dx0 dy0 dz0 [dx1 dy1 dz1 ...] \
  --uv-ray-norms n0 [n1 ...]
```

One direction or norm is repeated for all loop variables; otherwise one three-vector and one norm can be supplied per loop variable. If directions are given but no norm is supplied, the default starting norm is E_{cm} .

The same profiler was then run on the fixed ray matching the angular direction and radius ratio of the GL00 approach scan:

```
--uv-ray-directions
-0.66816740923660423 -0.71446756114601095 0.20757749710437134
-0.67621171790860379 -0.69980455265688968 0.23024183078620908
--uv-ray-norms 1 1.0221114989619468
```

The summed, not-per-orientation UV profiles are:

profile	entries	worst slope	r^2	bad entries	arb retry
random ray, all subsets	33	-0.997272	0.999997	0/33	0/33
tailored ray, double-free subsets	11	-0.812993	0.986953	11/11	11/11
tailored ray, single-free subsets	22	-1.000429	1.000000	0/22	0/22

Here “bad” means a fitted slope above the profiler warning threshold -0.9 . The random-ray result is the expected generic UV profile. With the tailored ray, all double-free subsets move to the same fitted behavior,

$$\log |I_{\text{prof}}| \simeq -0.812993 \log s + \text{const}, \quad r^2 = 0.986953,$$

and all of those double-free fits require the arbitrary-precision retry. Single-free subsets remain ordinary.

A follow-up scan continuously shrinks the plot-ray mismatch. Let $a = 1$ denote the GL00 plot ray above and $a = 0$ denote the exactly collinear, equal-radius ray with the first plot-ray direction used for both loop variables. The second direction is

$$\hat{v}(a) = \frac{(1-a)\hat{u} + a\hat{v}_{\text{plot}}}{|(1-a)\hat{u} + a\hat{v}_{\text{plot}}|}, \quad \rho(a) = 1 + a(\rho_{\text{plot}} - 1).$$

The table reports the summed double-free UV-profile slope. All 11 double-free LMB subsets give the same number for each row.

a	$\theta(\hat{u}, \hat{v})$	$\rho - 1$	$10^3 \rightarrow 10^5$	$10^3 \rightarrow 10^7$
1.000	2.817×10^{-2}	2.211×10^{-2}	-0.814	-0.931
0.500	1.408×10^{-2}	1.106×10^{-2}	-0.684	-0.881
0.200	5.633×10^{-3}	4.422×10^{-3}	-0.434	-0.776
0.100	2.817×10^{-3}	2.211×10^{-3}	-0.182	-0.661
0.050	1.408×10^{-3}	1.106×10^{-3}	+0.104	-0.516
0.020	5.633×10^{-4}	4.422×10^{-4}	+0.471	-0.289
0.010	2.816×10^{-4}	2.211×10^{-4}	+0.699	-0.101
0.005	1.408×10^{-4}	1.106×10^{-4}	+0.860	+0.092
0.000	0	0	+1.000	+1.000

A sparse extension to 10^9 gives slopes -0.964 for $a = 1$, -0.820 for $a = 0.1$, -0.611 for $a = 0.02$, -0.376 for $a = 0.005$, and $+1.000$ for $a = 0$. This is the expected non-uniform limit. For any fixed nonzero angular/radius mismatch, probing deep enough eventually resolves the relative momentum and pushes the fit back toward the generic negative UV behavior. As the mismatch is reduced, the crossover moves to larger scales, and over the finite UV windows relevant to the integrator the fitted slope degrades through zero and becomes positive. The exactly coincident GL00-direction ray remains positive because the relative hard momentum is kept fixed to the external shift instead of growing with the common UV scale.

The same two profiler runs were also repeated with `--per-orientation`. For the random ray, the double-free per-orientation profile remained clean: the worst double-free orientation slope was -0.988527 , with no arbitrary-precision retries and no slope above -0.9 . For the tailored ray, the issue is not isolated to a unique orientation. Among the 62 fitted orientations, 48 have at least one double-free entry above -0.9 . The strongest orientations are:

orientation	max slope	avg. double-free slope	bad entries	arb retry
0000+++--+	-0.614883	-0.614883	11/11	11/11
0000--++--	-0.615094	-0.615094	11/11	11/11
0000+++--+	-0.632973	-0.632973	11/11	11/11
0000--++--	-0.633049	-0.633049	11/11	11/11
0000---+--	-0.709478	-0.709478	11/11	11/11
0000++-+--	-0.709620	-0.709620	11/11	11/11

The repeated value across all 11 double-free LMB subsets is expected for this fixed ray because the ray is supplied in each LMB basis. Orientation 0000+++--+ is the worst fitted one, but it is only slightly worse than 0000--++--; the enhancement is therefore a broad orientation-sector feature rather than a single-orientation defect.

The z-axis exactly collinear equal-radius test ray, $(0,0,1)$ and $(0,0,1)$ with norms 1,1, produced missing fits rather than stable slopes. This was a direction-specific numerical special case, not the generic behavior of coincident rays; the GL00 plot-direction coincident ray gives the stable positive slopes shown above.

These fixed-ray results are not evidence for a generic uncanceled UV divergence. They instead expose a non-uniform, codimension relative-momentum ridge that the random-ray UV profile hides.

This is not in contradiction with the positive-slope event-weight plots. The UV profile uses momentum-space samples with `integrator_weight=1` and multiplies the raw integrand by only the scale-measure factor $s^{3n_{UV}}$. For the double-UV tests here this is s^6 . The radial approach plots, by contrast, show the density seen by the integrator after the spherical linear map. For one hard loop,

$$r = E_{\text{cm}} b \frac{x}{1-x}, \quad r^2 \frac{dr}{dx} \sim r^4,$$

whereas the UV shell-measure factor used by the profiler is r^3 per loop. Thus the plotted x -space weight contains one extra large-radius power per hard loop compared to the profiler's DOD diagnostic. A tailored-ray profiler slope near -0.813 can therefore correspond to a pointwise x -space weight that grows roughly like a positive power along the same radial approach. When the rays are tuned even closer together, the profiler itself finds zero or positive slopes along that forced line. The transverse angular and radius-ratio volume is not represented by this one-dimensional profile, so the positive forced-ray DOD identifies an integrable relative singularity and a variance problem rather than a generic UV divergence of the full integral.

The profiler JSON and logs for the default, plot-ray, simplified split-ray, exactly collinear, and per-orientation tests are saved in the report data directory under the `uv_profile.*` filenames.

7 Current long-run integration checks

The current all-LMB one-million-sample diagnostic, using Monte Carlo over LMB channels, kinematics A, helicity $+-$, $m_{UV} = 9.188$, and $\mu_R = 91.188$, gave:

component	value	error	relative error
real	-6.8192×10^{-6}	4.7659×10^{-6}	69.9%
imag	-5.4228×10^{-6}	3.3372×10^{-6}	61.5%

The largest signed weights in that run were:

component/sign	max evaluation	LMB channel
real +	$+5.3591 \times 10^{-1}$	(5, 9)
real -	-4.3118	(5, 10)
imag +	$+5.4193 \times 10^{-1}$	(5, 9)
imag -	-2.5294	(5, 10)

A longer Monte-Carlo-over-LMB run with 90 M samples and 20 cores completed in 739.3 s. It did not reach the requested sub-10% Monte-Carlo error:

component	value	error	relative error	max-weight impact
real	-4.4150×10^{-7}	4.2964×10^{-7}	97.3%	0.791
imag	-1.2004×10^{-6}	3.0318×10^{-7}	25.3%	0.134

The largest signed weights in this run moved to LMB channel (5, 7). The largest real negative event was -31.4201, and the largest imaginary negative event was -14.4356, both at the same sampled point. The adaptive grid probability for channel (5, 7) was about 42.2%, but the continuous relative ridge still dominated the variance.

An explicit summed-LMB run with 10 M samples was run for a comparable wall time, 740.9 s. Summing removes the discrete LMB sampling noise but makes each sample roughly an order of magnitude more expensive. At equal wall time it also did not converge:

component	value	error	relative error	max-weight impact
real	$+6.6162 \times 10^{-8}$	3.2102×10^{-7}	485.2%	1.513
imag	-1.1342×10^{-6}	4.4162×10^{-7}	38.9%	0.190

The summed-channel run lowers the absolute largest sampled weights compared to the MC-over-LMB run, but the real part is close enough to zero that its relative error remains very large. These results show that the default $\alpha = 3$ LMB multichannel setup did not reach stable sub-10% errors in the tested wall time. They should not be read as proof that LMB multichanneling is irrelevant, or that no choice of α can help; the unresolved issue is the continuous relative-momentum ridge left after the discrete OSE reweighting.

The saved run artifacts are in the report data directory, with separate JSON and log files for the 90 M MC-over-LMB run and the 10 M summed-LMB run.

8 What the LMB multichannel prefactor does

The generated all-LMB setup contains channels where edge 4 is a basis edge, and the LMB multichanneling code does use this information. For a channel c with basis edges $e \in c$, the channel weight is

$$w_c = \left(\prod_{e \in c} E_e \right)^{-\alpha}, \quad p_c = \frac{w_c}{\sum_j w_j}. \quad (12)$$

With `lmb_channels = "monte_carlo"`, one LMB is sampled with this normalized weight. With `lmb_channels = "summed"`, all generated LMBs are evaluated with the same normalized prefactors. The default value is $\alpha = 3$, and the flat `[sampling]` parser now exposes this as an `alpha` field so that it can be scanned directly.

I evaluated this normalized prefactor on the same ray family used in the fixed-ray UV-profile scan. The table reports the raw probability assigned to the problematic (5, 10) interpretation and the total probability assigned to channels containing edge 4:

a	R	α	E_4/R	$p_{(5,10)}$	$\sum_{4 \in c} p_c$
1.000	10^3	1	2.041×10^{-1}	3.050×10^{-2}	0.854066
1.000	10^3	3	2.041×10^{-1}	1.621×10^{-3}	0.992885
1.000	10^7	1	3.566×10^{-2}	5.814×10^{-3}	0.970933
1.000	10^7	3	3.566×10^{-2}	7.726×10^{-6}	0.999961
0.100	10^7	1	3.584×10^{-3}	5.959×10^{-4}	0.997020
0.100	10^7	3	3.584×10^{-3}	7.687×10^{-9}	1.000000
0.020	10^7	1	7.206×10^{-4}	1.200×10^{-4}	0.999400
0.020	10^7	3	7.206×10^{-4}	6.240×10^{-11}	1.000000
0.005	10^7	1	1.843×10^{-4}	3.072×10^{-5}	0.999846
0.005	10^7	3	1.843×10^{-4}	1.044×10^{-12}	1.000000
0.000	10^7	1	1.973×10^{-5}	3.287×10^{-6}	0.999984
0.000	10^7	3	1.973×10^{-5}	1.279×10^{-15}	1.000000

The conclusion is that the OSE prefactor is doing something very strong. On the near-collinear rays, default $\alpha = 3$ assigns essentially all normalized channel weight to LMBs containing edge 4 and suppresses the direct (5, 10) interpretation by many orders of magnitude. Increasing α would make this channel selection even sharper; decreasing it would make the channel selection smoother. Therefore the previous statement that LMB MC merely samples an LMB, or that it cannot be sufficient by construction, would be too strong.

However, the OSE prefactor is not the same object as the balance-heuristic density of the actual channel maps. For channel maps $y = \phi_c(x_c)$ from unit-cube variables to physical loop momenta, a standard multichannel denominator would use

$$p_c(y) \propto \frac{\pi_c}{J_c(y)}, \quad J_c(y) = \left| \det \frac{\partial \phi_c}{\partial x_c} \right|_{x_c = \phi_c^{-1}(y)}, \quad (13)$$

up to the discrete prior π_c . The original OSE mode instead computes the partition in Eq. (12): it reinterprets the sampled momenta in the selected LMB, keeps the sampled conformal-map Jacobian, and multiplies by the normalized OSE product. Thus the OSE denominator knows which LMB energies are small, but it does not know the actual conformal-map density induced by each competing LMB parameterization.

I added a two-channel test to the standalone proxy to isolate this distinction. The two channels are the bad direct interpretation (u, v) and an explicit relative interpretation

$$P_{\text{common}} \simeq \frac{u+v}{2}, \quad q = v - u + Q.$$

For a spherical linear conformal map the one-vector Jacobian used in `parameterize3d` is

$$J_3(r) = 4\pi r^2 \frac{(E_{\text{cm}}b + r)^2}{E_{\text{cm}}b}.$$

The inverse-Jacobian balance denominator was compared to OSE partitions with $\alpha = 1, 3, 4$.

path	raw slope	OSE $\alpha = 1$	OSE $\alpha = 3$	OSE $\alpha = 4$	inverse J , linear
fixed $ q = 0.2$, varying R	+1.00	+0.00	-2.00	-3.00	-3.00
fixed $ q /R = 10^{-3}$	-1.00	-1.00	-1.00	-1.00	-1.23
fixed $R = 10^6$, varying $ q $	-1.16	-0.16	+1.84	+2.84	+0.85

Here “slope” means the fitted slope of the bad direct-channel contribution after multiplying by the corresponding partition factor. On the strict locked-ray path with fixed small q , the raw bad-channel weight grows like R . The old OSE factor with $\alpha = 3$ already changes that contribution to R^{-2} . An inverse-Jacobian denominator for the linear map gives R^{-3} , numerically close to OSE $\alpha = 4$. At $R = 10^7$ and $|q| = 0.2$, the bad direct-channel probabilities are

$$p_{\text{bad}}^{\text{OSE}, \alpha=3} = 8.0 \times 10^{-24}, \quad p_{\text{bad}}^{\text{OSE}, \alpha=4} = 1.6 \times 10^{-31}, \quad p_{\text{bad}}^{1/J, \text{linear}} = 3.3 \times 10^{-26}.$$

For the logarithmic radial map, the inverse-Jacobian probability is even smaller in the deep UV because $J_3(r)$ grows exponentially with r/E_{cm} in that map; it underflows below double precision already in the large- R part of this scan. The OSE denominator has no way to know about that map-dependent density. Figure 9 compares these partition factors in the standalone proxy.

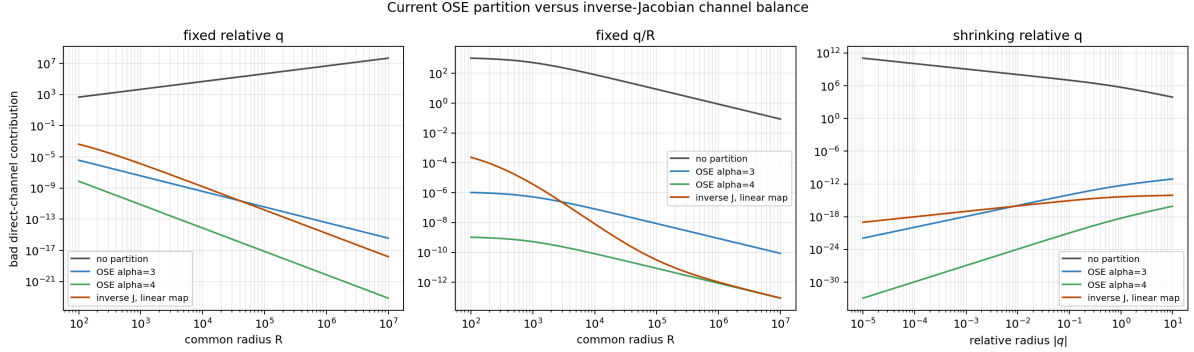


Figure 9: Standalone proxy comparison of the current OSE partition and an inverse-Jacobian balance partition. The OSE $\alpha = 3$ factor already suppresses the strict small-relative-momentum direct channel strongly. The inverse-Jacobian denominator is more map-faithful and gives one additional power of R for the linear map on the fixed- q locked ray, while the logarithmic map would suppress the bad direct channel even more strongly in the far UV.

8.1 Pointwise inspect check of LMB multichanneling

I also checked the problematic GL00 region directly with `inspect`, using kinematics A and helicity $+-+-$. The current `examples/cli/aa_aa/2L` state is not an all-LMB state: `display integrand` shows only one active LMB channel, channel 0 with basis (5, 10). Therefore, in that state, switching `lmb_multichanneling=true` and `lmb_channels="monte_carlo"` does not actually average over all LMBs; it evaluates the same single selected LMB channel.

For the all-LMB regenerated state, `display integrand` shows eleven active channels:

$$(5, 10), (4, 10), (5, 9), (4, 9), (5, 8), (4, 8), (5, 7), (4, 7), (5, 6), (4, 6), (4, 5).$$

The table below compares the displayed inspect weight, i.e. the integrand times the continuous parameterization Jacobian. These are the weights seen before the additional discrete-channel normalization present in the Monte-Carlo-over-LMB estimator.

setup	point	Re weight	Im weight	weight
LMB off	point A	$-2.63307595 \times 10^{-1}$	$+2.16943924 \times 10^0$	2.18535982×10^0
single active LMB channel 0, (5, 10)	point A	-4.20480647×10^1	$+8.23757741 \times 10^1$	9.24867985×10^1
all LMBs, explicit summed channels	point A	$-1.84215087 \times 10^{-3}$	$+3.52144262 \times 10^{-3}$	$3.97417639 \times 10^{-3}$
LMB off	channel-6 max-weight point	$+7.74559448 \times 10^{-4}$	$-8.69620692 \times 10^{-4}$	$1.16455248 \times 10^{-3}$
LMB channel 6, (5, 7)	channel-6 max-weight point	-3.05497125×10^0	-1.40357390×10^0	3.36197398×10^0

This does not mean that the multichannel denominator only acts when an edge-4 channel is selected. The intended balance mechanism is global at the sampled physical point: even if the sampled chart is (5, 10) or (5, 7), the denominator

$$\sum_c \prod_{e \in c} E_e^{-\alpha}$$

contains the competing edge-4 charts. The question is therefore why this global denominator is not sufficient.

For point A, interpreted in the bad (5, 10) chart, the local normalized prefactor for the selected channel is already small,

$$p_{(5,10)}^{\text{OSE}, \alpha=3} = 8.46 \times 10^{-6}.$$

The relevant edge scales are

$$|e_4| = 1.78 \times 10^3, \quad E_4 = 1.79 \times 10^3, \quad |e_5| = 4.82 \times 10^4, \quad E_{10} = 4.93 \times 10^4.$$

So the OSE denominator is doing the expected qualitative thing in the deep UV approach: the edge-4 charts dominate the denominator and suppress the bad chart. The remaining problem is quantitative and structural: the raw bad-chart density is large enough that this OSE-product suppression does not flatten the estimator, and the OSE product is not the true density of the competing maps.

I then ran a controlled comparison in the same all-LMB regenerated state, with the same seed and 10 M samples:

setup	Re	Im	largest $ w $	top channel
LMB off	$-5.55(8.03) \times 10^{-8}$	$-7.19(0.85) \times 10^{-7}$	2.69×10^{-1}	none
all LMBs, MC over LMBs	$-4.94(7.22) \times 10^{-7}$	$-3.31(10.49) \times 10^{-7}$	7.22	(5, 7)

In this controlled run, and also in the 90 M all-LMB run, the largest weights are in channel (5, 7). This channel is precisely a chart in which the edge-4 momentum is a difference of two sampled basis momenta:

$$e_5 = -K_0 + K_1 + P_0, \quad e_7 = K_1 + P_0, \quad e_4 = K_0 = e_7 - e_5.$$

The necessary edge-4 LMBs are present, for example (4, 7) and (4, 5). What fails is not channel enumeration; it is the feature used in the balance denominator.

At the largest (5, 7) points the edge-4 *three-momentum* is small, but edge 4 is a massive top edge, so its OSE does not become small:

point	$ e_4 $	E_4	selected-channel prefactor	largest sampled channel
10 M all-LMB MC max	8.71	173.22	8.78×10^{-2}	(5, 7)
90 M all-LMB MC max	7.47	173.16	1.17×10^{-1}	(5, 7)

Thus the OSE partition barely sees the small relative three-momentum in this moderate-radius ridge: $E_4 \simeq m_t$ is just a mass scale. This is exactly where an inverse-Jacobian or relative-momentum density would differ from the current OSE proxy. For a spherical linear map, a chart that samples the relative vector $q = e_4$ directly has a Jacobian factor proportional to q^2 near $q = 0$, so its inverse density is enhanced like $1/q^2$ even for a massive edge. The OSE factor $E_4^{-\alpha}$ has no corresponding enhancement because $E_4 = \sqrt{|q|^2 + m_t^2}$.

This answers the inverse-Jacobian question as follows. Replacing the OSE-product denominator by the actual inverse-Jacobian channel density is more principled and helps in exactly the regimes where the conformal map itself controls the rare-event density, especially for logarithmic mapping and large common radius. It also makes the channel balance automatically adapt to E_{cm} and b . It is the right diagnostic for the massive-edge issue above, because it is sensitive to the small spatial relative variable even when the corresponding OSE is pinned by the mass. The OSE factor is therefore not useless; it suppresses the deep-UV bad chart by many orders of magnitude. But it is not the same as the balance density of the actual channel maps, and for the massive relative edge it misses the most important local variable, $|e_7 - e_5|$.

This explains why the longer all-LMB Monte-Carlo run assigned large probability to channel (5, 7), about 42.2%, and sizable probabilities to channels such as (5, 10), about 15.8%, while individual edge-4 channels were only about 1.4% each. The adaptive procedure sees the contribution and variance of the existing channels, but it does not infer a new relative-momentum

map. The practical recommendation is therefore: disable the LMB-selection heuristic when studying these graphs so that all LMB channels are actually available; keep explicit LMB summation as a diagnostic baseline; and add a continuous relative-momentum channel for hard-hard configurations where an internal momentum such as $k - l + Q$ becomes small while both loop momenta are UV. The multichannel denominator should then use the actual densities of the competing maps, or at least an inverse-Jacobian approximation to them, instead of relying only on the OSE product. An α scan remains useful now that the setting is exposed in `[sampling]`, but tuning α alone cannot create the missing relative variable.

8.2 Implemented inverse-Jacobian LMB weights

The code now supports `lmb_channel_weight = "inverse_jacobian"` in the flat `[sampling]` settings. For a selected LMB channel s and a physical loop-momentum point y , the new prefactor is

$$\frac{J_s(y)^{-1}}{\sum_c J_c(y)^{-1}}, \quad (14)$$

where each J_c^{-1} is obtained by applying the inverse parameterization of channel c to the same physical point. The old OSE mode remains available as `lmb_channel_weight = "ose"`, but the inverse-Jacobian mode is now the default because it matches the balance-density interpretation of the competing LMB maps.

For GL00, with kinematics A, helicity $+-+-$, $m_{UV} = 9.188$, $\mu_R = 91.188$, spherical linear mapping, summed orientations, and all LMB channels enabled, the direct comparison is:

LMB treatment	samples	Re	Im	Im rel. error	max weight	avg time [μ s]
summed, OSE	1 M	-0.850 ± 0.783	-0.232 ± 1.024	441.6%	0.467	1368.9
summed, inverse J	1 M	-0.218 ± 0.180	-0.610 ± 0.182	29.9%	0.00903	1396.5
MC, OSE	10 M	-0.529 ± 0.772	-0.354 ± 1.122	317.3%	7.719	130.0
MC, inverse J	10 M	0.031 ± 0.125	-0.757 ± 0.125	16.5%	0.0625	130.0
MC, inverse J	50 M	-0.034 ± 0.056	-0.690 ± 0.056	8.12%	0.104	131.0

The values in the table are in units of 10^{-6} except for the last two columns. The real part remains compatible with zero, so its relative error is not a useful convergence criterion here. The imaginary part reaches the requested sub-10% precision at 50 M samples with LMB Monte Carlo. The largest sampled weights are reduced by roughly two orders of magnitude relative to the OSE Monte-Carlo run, and by more than two orders relative to the old worst events. The remaining largest points no longer sit on the original equal-radius collinear double-UV ray; the most severe residual weights are moderate finite-radius or unequal-radius configurations.

I also checked the one-loop $\gamma\gamma \rightarrow \gamma\gamma$ amplitude with the same inverse-Jacobian LMB mode. The reference quoted in `examples/cli/aa_aa/1L/aa_aa.toml` for kinematics A and helicity $+-+-$ is $-6.9028625224992605 \times 10^{-7}$. Integrated-UV generation requires the development shell because `form/tform` are not on the bare shell `PATH`. With the compiled 1L state generated in that environment, both inverse-Jacobian checks are statistically compatible with the reference:

setup	samples	Re	Im	target pull
graph MC, LMB summed	5 M	-0.283 ± 0.820	0.186 ± 0.814	0.50σ
graph summed, LMB summed	5 M	-0.088 ± 0.781	0.636 ± 0.781	0.77σ

The Re and Im columns in this one-loop table are also in units of 10^{-6} . The 1L validation is therefore a compatibility check rather than a precision measurement: the sampled errors are still larger than the reference value, but there is no visible bias from the inverse-Jacobian channel prefactor. The raw artifacts for these checks are stored as `gl00_inverse_jacobian.lmbmc_50M.result.json` and `aa_aa_1l_pmpm_inverse_jacobian_5M.*result.json` in the report data directory.

9 Other $\gamma\gamma \rightarrow \gamma\gamma$ two-loop graphs

I reran the full selected two-loop graph set GL00, GL01, GL03, GL05, GL06, GL08, GL12, GL14, GL16, GL17, and GL18. Each graph was generated in a fresh state with integrated UV enabled and a 100 GiB virtual-memory cap. All eleven labels now complete generation and the standard random-ray UV profile. The profiling and integration runs used kinematics A, helicity $+-+-$, disabled thresholds, $\mu_R = 91.188$, and $m_{UV} = 9.188$.

The integration campaign used 20 cores, one graph at a time, summed orientations, generated LMB multichanneling, and Monte-Carlo sampling over LMB channels rather than explicit summation over LMB channels. Each graph was run for a 10-minute wall-time window with `integrator.n_start=100000`, `n_increase=0`, and `n_max=1000000000`. The timeout exit code is therefore expected; the table below uses the final completed iteration written before termination.

graph	feyngen [s]	generation [s]	peak RAM [GiB]	UV entries	worst UV slope	slopes > -0.9
GL00	8	64	13.01	33	-0.99727	0
GL01	16	86	16.32	33	-0.99544	0
GL03	8	74	14.46	33	-0.99575	0
GL05	24	86	13.80	33	-0.99837	0
GL06	7	24	2.83	42	-0.99572	0
GL08	30	42	2.19	45	-0.99796	0
GL12	11	22	1.87	45	-0.99545	0
GL14	2	17	2.77	42	-0.99908	0
GL16	27	39	2.27	45	-0.99940	0
GL17	10	26	2.83	42	-0.99848	0
GL18	6	21	2.32	42	-0.99765	0

All random-ray UV profiles are therefore clean in this diagnostic sense: every fitted slope is negative with comfortable margin, no fitted slope is above -0.9 , and no arbitrary-precision retry was needed. This remains consistent with the GL00-specific observation above: the UV profiler probes generic rays well, while the integrator can still see rare high-weight relative-momentum ridges.

graph	evals $[10^6]$	avg time [total/int/eval μ s]	Re $[10^{-6}]$	Im $[10^{-6}]$	10-minute status
GL00	43.5	158.5/73.5/58.7	-0.991 ± 1.048 (105.8%)	-0.745 ± 1.656 (222.2%)	not converged
GL01	44.4	159.9/73.7/57.0	-2.656 ± 2.092 (78.7%)	1.159 ± 3.721 (321.0%)	not converged
GL03	47.2	145.6/68.1/57.1	0.249 ± 0.703 (282.2%)	1.383 ± 0.932 (67.3%)	not converged
GL05	45.5	150.8/70.3/57.5	0.579 ± 1.673 (289.1%)	2.193 ± 2.122 (96.8%)	not converged
GL06	47.6	129.0/59.5/46.8	-0.008 ± 0.035 (456.6%)	-1.135 ± 0.038 (3.3%)	imaginary part only
GL08	53.8	126.5/54.1/26.3	-0.857 ± 0.846 (98.8%)	-3.435 ± 2.302 (67.0%)	not converged
GL12	58.0	114.0/49.1/26.1	1.476 ± 1.580 (107.0%)	0.505 ± 0.143 (28.2%)	not converged
GL14	43.9	143.7/65.3/47.4	0.009 ± 0.038 (404.3%)	4.234 ± 0.043 (1.0%)	imaginary part only
GL16	49.3	145.8/61.5/26.6	0.325 ± 0.252 (77.4%)	-3.965 ± 0.234 (5.9%)	imaginary part only
GL17	45.3	143.9/65.4/47.3	-0.046 ± 0.040 (87.7%)	4.155 ± 0.044 (1.1%)	imaginary part only
GL18	45.6	141.6/64.4/46.9	-0.033 ± 0.047 (144.6%)	-1.158 ± 0.053 (4.6%)	imaginary part only

No complete graph reached sub-10% Monte-Carlo uncertainty in both real and imaginary parts in the 10-minute, 20-core, LMB-channel-MC setup. The behavior separates into two groups. GL00, GL01, GL03, GL05, GL08, and GL12 remain high-variance in both components. GL06, GL14, GL16, GL17, and GL18 have a stable imaginary component at the percent level, but a real part compatible with zero and therefore a large relative error. Thus the graph-level convergence problem is not only a GL00 issue, although the catastrophic double-UV ridge documented in the earlier sections is the clearest mechanistic example.

The compact campaign summary and per-graph final UV-profile and integration JSON files are archived in the report data directory under `aa_aa_21_all_graphs_10min_lmbmc`.

9.1 Max-weight geometry and fixed-ray UV profiles

For each graph, I then took the largest absolute entry in the stored `max_weight_info`. The spherical unit-cube coordinates were decoded to the two sampled LMB ray directions and radii. I ran two additional UV profiles from those data:

1. an exactly locked profile, where the two UV rays are made exactly parallel, or anti-parallel when the recorded LMB directions have opposite sign, and are assigned the same starting norm;
2. an actual-angular profile, where the two recorded angular directions are used as-is, again with equal starting norms.

The first profile tests the strict equal-norm relative-momentum ray. The second profile checks whether the finite max-weight point itself already sits on the asymptotic ray. In all cases the usual random-ray UV profile is clean, so the table below reports only the largest fitted slope from each follow-up.

graph	max eval	angle [deg]	r_1/r_0	exact locked slope	actual-angular slope	exact nonnegative fits	interpretation
GL00	-42.9	0.954	1.156	1.000	-1.000	11	confirmed GL00-type ray
GL01	161.5	1.412	0.947	-1.000	-1.000	0	finite near-collinear peak
GL03	-16.3	9.270	1.082	1.000	-1.000	11	confirmed GL00-type ray
GL05	85.0	10.901	0.967	-1.000	-1.000	0	finite near-collinear peak
GL06	-0.536	2.477	0.960	1.000	-0.999	14	confirmed, small residue
GL08	-123.3	0.771	1.016	-0.964	-0.965	0	borderline finite peak
GL12	91.2	179.458	1.040	2.000	-0.962	15	confirmed anti-parallel ray
GL14	0.582	0.434	1.004	1.000	-1.000	14	confirmed, small residue
GL16	7.13	178.023	1.044	2.000	-0.974	15	confirmed anti-parallel ray
GL17	0.812	7.256	0.992	1.000	-1.000	14	confirmed, small residue
GL18	-1.47	179.676	0.985	1.000	-1.000	14	confirmed anti-parallel ray

This confirms that the GL00 diagnostic is not isolated. The same strict equal-norm double-UV ray is exposed by the UV profiler for GL00, GL03, GL06, GL12, GL14, GL16, GL17, and GL18. For GL12, GL16, and GL18, the same condition appears as anti-parallel rays in the sampled LMB basis; this is the same relative-momentum mechanism with a different sign convention in the chosen basis. The positive slopes always occur in the full double-UV subset where both LMB edges are free, for example the `subset=2` entries `free=(5,10)`, `free=(4,10)`, and analogous pairs in the generated LMB list.

The actual-angular fixed profiles are negative even when the recorded max-weight point is less than one degree from collinearity. This is important: the profiler only sees the positive slope when the two UV rays are locked exactly onto the relative-momentum ray. A small opening angle or radius mismatch restores a negative asymptotic slope, while the finite coefficient can still be large enough to dominate a Monte-Carlo iteration.

GL01, GL05, and GL08 are therefore different from the confirmed group in this diagnostic. Their largest weights are still near-collinear and near equal-radius, but the fixed-ray UV profile does not show a nonnegative asymptotic slope. I pushed GL01 and GL05 to `max-scaling=15`; both stayed at slope -1 . GL08 is the borderline case: it gives a mildly degraded double-UV slope near -0.964 at `max-scaling=9` and -0.937 at `max-scaling=15`, but a deeper `f128` run to `max-scaling=21` returns to -1.000 . I therefore classify these three as large finite near-collinear peaks rather than confirmed GL00-type positive-DOD rays.

The detailed fixed-ray summary is archived as `aa_aa_21_all_graphs_10min_lmbmc/max_weight_uv_diagnostic_summary.json`.

9.2 Full-amplitude run with explicit summed LMB channels

I then generated all eleven two-loop graphs together with feynngen symmetry factors and integrated the full $\gamma\gamma \rightarrow \gamma\gamma$ amplitude at kinematics A and helicity $+-+ -$. The run used

graph Monte Carlo, summed orientations, explicit summation over all generated LMB channels, `lmb_channel_weight="inverse_jacobian"`, the spherical linear map, $m_{UV} = 9.188$, and $\mu_R = 91.188$. After 80 M samples on 20 cores the result was:

component	value	error	relative error	max-weight impact
real	-1.4573×10^{-7}	1.4224×10^{-7}	97.6%	7.04%
imag	-1.3755×10^{-6}	1.4245×10^{-7}	10.36%	1.33%

The real part is compatible with zero in this helicity/kinematics setup, so the imaginary part is the useful convergence target. The current best full-run imaginary uncertainty is just above the requested 10% level and still an order of magnitude away from a 1% target. The observed $1/\sqrt{N}$ scaling would require about 8 B samples for a brute-force 1% imaginary-part error with this exact setup.

I also recomputed the graph-level variance allocation from the 80 M grid entries. The adaptive graph probabilities are already essentially optimal for this estimator: at fixed total sample count the best possible graph-only allocation would reduce the combined imaginary error by only 2.8×10^{-4} relative, and it gives the same 8.47 B sample estimate for a 1% imaginary-part error. Retuning the graph Monte Carlo probabilities therefore cannot solve the current variance bottleneck.

The all-graph generation state reports a peak resident-memory scale of 60.2 GB, or about 56.1 GiB. The per-graph generation times in that state are 66.0 s for GL00, 77.1 s for GL01, 77.3 s for GL03, 62.9 s for GL05, and between 11.7 s and 17.3 s for the remaining seven graphs.

The higher-stat graph grid at 80 M samples is given in Table 1. This table is regenerated from `data/aa_aa_2l_all_graphs_inverse_jacobian_sumlmb_80M/integration_result_80M.json`; the entries are the top-level graph dimension of the monitored integration.

graph	processed samples [10^6]	graph prob. [%]	Re [10^{-6}]	Im [10^{-6}]	Im rel. error
GL00	6.56	8.08	-0.036 ± 0.040	-0.728 ± 0.040	5.49%
GL01	9.93	12.39	-0.002 ± 0.051	-3.994 ± 0.050	1.25%
GL03	14.25	17.95	-0.016 ± 0.058	$+3.937 \pm 0.060$	1.53%
GL05	5.89	7.31	$+0.026 \pm 0.038$	-0.701 ± 0.038	5.44%
GL06	8.97	11.22	-0.082 ± 0.048	-2.284 ± 0.047	2.07%
GL08	3.21	3.96	$+0.077 \pm 0.028$	-1.301 ± 0.028	2.15%
GL12	2.93	3.62	$+0.002 \pm 0.026$	$+1.272 \pm 0.027$	2.10%
GL14	6.26	7.71	-0.052 ± 0.040	$+4.184 \pm 0.039$	0.93%
GL16	2.97	3.64	$+0.016 \pm 0.026$	-3.598 ± 0.027	0.74%
GL17	6.04	7.18	$+0.024 \pm 0.037$	$+4.095 \pm 0.037$	0.90%
GL18	12.99	16.94	-0.093 ± 0.060	-2.282 ± 0.059	2.61%

Table 1: Per-graph contributions from the 80 M-sample all-graph run with graph Monte Carlo, summed orientations, summed LMB channels, and inverse-Jacobian LMB weights.

The average total runtime was 1.40 ms per sample per core, with about 0.71 ms in the integrand and 0.66 ms in the evaluator. The combined run stayed numerically stable: *f*64 was used for 99.9984% of samples, *f*128 for 0.00163%, and the nan-or-unstable rate was $2.5 \times 10^{-6}\%$.

9.3 Final committed-card production run

For the final wrap-up, I updated `examples/cli/aa_aa/2L/aa_aa.toml` so that the default command builds the full two-loop amplitude with `feynngen` and then runs `generate_integrands` followed by `integrate_diagrams`. The default integration setup is the best robust setup identified above: kinematics A, helicity $+-+-$, $m_{UV} = 9.188$, $\mu_R = 91.188$, graph Monte Carlo, summed orientations, explicit summed LMB channels, inverse-Jacobian LMB channel weights, spherical coordinates, the linear radial map, 20 integration cores, and 1 M samples per iteration. The Havana training phase is explicitly set to the imaginary part with

`integrator.integrated_phase="imag"` both in the `integrate_diagrams` command block and in the card’s default runtime settings. Because vakint tensor reduction calls FORM as an external program and the local `./gamma_loop` wrapper does not itself enter the Nix development shell, the card also pins `form_exe_path` to the Nix-provided FORM executable used in this environment.

Using the already generated all-graph state with these committed settings, I ran the final 25 M-sample production checkpoint on 20 cores. It finished after 34 minutes and 44 seconds. The total amplitude was

component	value	error	relative error	χ^2
real	-9.34×10^{-8}	2.62×10^{-7}	281%	28.16
imag	-1.3638×10^{-6}	2.592×10^{-7}	19.0%	10.15

The corresponding displayed “sum” entry is $\text{Re} = (-1.1 \pm 2.6) \times 10^{-7}$ and $\text{Im} = (-1.37 \pm 0.25) \times 10^{-6}$. The real part is again compatible with zero, while the imaginary part agrees with the previous 80 M-sample central value within the quoted Monte-Carlo error. The higher-stat per-graph values are the 80 M entries in Table 1; the table below is kept only as the direct check of the final committed-card run. The 30-minute run therefore confirms the central value but does not reach a 1% total-amplitude error; the total remains dominated by cancellations between graph contributions.

The per-graph top-level discrete-grid entries from this final run were:

graph	processed samples [10 ⁶]	Re [10 ⁻⁶]	Im [10 ⁻⁶]	Im rel. error
GL00	2.10	-0.133 ± 0.075	-0.629 ± 0.074	11.74%
GL01	3.18	$+0.157 \pm 0.093$	-4.036 ± 0.091	2.24%
GL03	4.19	-0.188 ± 0.108	$+3.817 \pm 0.106$	2.78%
GL05	1.91	-0.032 ± 0.069	-0.769 ± 0.069	8.97%
GL06	2.81	-0.108 ± 0.088	-2.238 ± 0.086	3.83%
GL08	1.05	-0.028 ± 0.050	-1.193 ± 0.050	4.21%
GL12	0.97	$+0.091 \pm 0.053$	$+1.309 \pm 0.050$	3.79%
GL14	2.05	$+0.057 \pm 0.072$	$+4.187 \pm 0.071$	1.69%
GL16	0.95	$+0.026 \pm 0.046$	-3.627 ± 0.048	1.32%
GL17	1.79	-0.053 ± 0.068	$+4.215 \pm 0.066$	1.57%
GL18	4.01	$+0.105 \pm 0.107$	-2.402 ± 0.106	4.42%

The average runtime was 1.401 ms per sample per core, with 0.710 ms in the integrand and 0.658 ms in the evaluator. Precision escalation was negligible: 99.9984% of evaluations ran in *f*64, 0.00162% in *f*128, none in Arb, and the reported nan-or-unstable rate was zero. The largest signed weights stayed below one in absolute value: +0.646 for real, −0.468 for real, +0.388 for imaginary, and −0.317 for imaginary. The positive and negative maxima occurred in GL12 and GL00, respectively, but they are ordinary $\mathcal{O}(1)$ samples, not a resurgence of the earlier double-UV large-weight ridge. The checkpoint and log are archived in `data/final_30m_all_graphs_inverse_jacobian_sumlmb/`.

I also tested several alternatives against this setup:

- summing over graphs as well as LMBs gives cleaner discrete sampling but costs about 15 ms per sample per core; at 1 M samples it gave $\text{Im} = (-1.92 \pm 1.10) \times 10^{-6}$, so it is not competitive at fixed wall time;
- hyperspherical coordinates with summed inverse-Jacobian LMB channels did not finish a 1 M-sample checkpoint before being stopped, making the current implementation too slow for this target;

- a global radial power map $r = E_{\text{cm}} b(x/(1-x))^p$ was implemented and tested. On the GL03 fixed ray, $p = 2$ flattens the asymptotic radial slope, while $p = 3, 4$ make it decreasing. In full integration this does not yet improve the variance: $p = 1.5$ and $p = 1.75$ are stable but have about the same 1 M-sample absolute error as the linear map, and $p = 2$ samples much deeper UV points where the double-precision cancellation is unreliable unless high loop-momentum norms are forced to arbitrary precision. Smaller b values keep the $p = 2$ ray flat but increase the weight coefficient on the same physical ray;
- Monte Carlo over LMB channels with inverse-Jacobian weights is about an order of magnitude cheaper per sample, but the 1 M-sample imaginary error was 5.61×10^{-6} , far worse than the explicit summed-LMB setup.

Several one-iteration integrator controls were also tried with graph Monte Carlo, summed orientations, summed LMBs, inverse-Jacobian LMB weights, and the linear spherical map:

control	Im value	Im error	avg time per sample/core
max_prob_ratio=300	$+0.95 \times 10^{-6}$	2.24×10^{-6}	1.33 ms
n_bins=128	$+0.95 \times 10^{-6}$	2.24×10^{-6}	1.31 ms
train_on_avg=true	-7.62×10^{-6}	2.25×10^{-6}	1.32 ms
max_prob_ratio=1000	-7.62×10^{-6}	2.25×10^{-6}	1.37 ms

The paired equal central values are from paired one-iteration seeds, so these rows should not be overinterpreted statistically. The useful conclusion is that none of these grid controls changes the absolute error scale relative to the baseline 1 M-sample run.

9.4 80M max-weight ridge recheck

The largest signed weights in the 80 M full-amplitude run occurred in GL18, GL01, GL03, and GL17. I first verified from `display integrand` that the diagrams have the full generated LMB sets enabled: GL01 and GL03 have 11 channels, while GL17 and GL18 have 14. In each case the edge-4 channels are present, for example (4, 10), (4, 9), (4, 8), (4, 7), (4, 6), and (4, 5) where applicable. Therefore the latest max-weight ridges are not caused by accidentally leaving the LMB heuristic on.

I then reran direct `inspect` scans from each recorded max-weight point, rescaling only the two radial coordinates deeper into the same hard-hard direction. The table reports the fitted log-log slope of the last four radial points and the absolute inspected weight at the recorded point:

point	mode	slope	abs. weight at point	abs. weight deepest
GL01 real −	no LMB	1.0006	0.0204	0.327
GL01 real −	OSE	1.0003	0.0266	0.426
GL01 real −	inverse J	1.0002	0.0203	0.326
GL03 imag +	no LMB	1.0000	0.376	6.01
GL03 imag +	OSE	1.0000	0.305	4.88
GL03 imag +	inverse J	1.0000	0.303	4.84
GL17 imag −	no LMB	1.0966	0.0681	1.36
GL17 imag −	OSE	1.0945	0.0628	1.25
GL17 imag −	inverse J	1.0980	0.0628	1.26
GL18 real +	no LMB	1.0000	0.638	10.2
GL18 real +	OSE	1.0000	0.360	5.76
GL18 real +	inverse J	1.0000	0.363	5.80

The inverse-Jacobian LMB prefactor is therefore active, but it does not change the hard-hard radial power on these locked approaches. It reduces the normalization for GL03 and GL18, leaves GL01 nearly unchanged, and leaves GL17 essentially unchanged. A logarithmic map combined with inverse-Jacobian LMB weighting was also tested on the same points and made the locked scans worse, with slopes between 1.65 and 1.77 and much larger absolute weights. Two representative radial rescans from the 80 M max-weight points are shown in Figure 10.

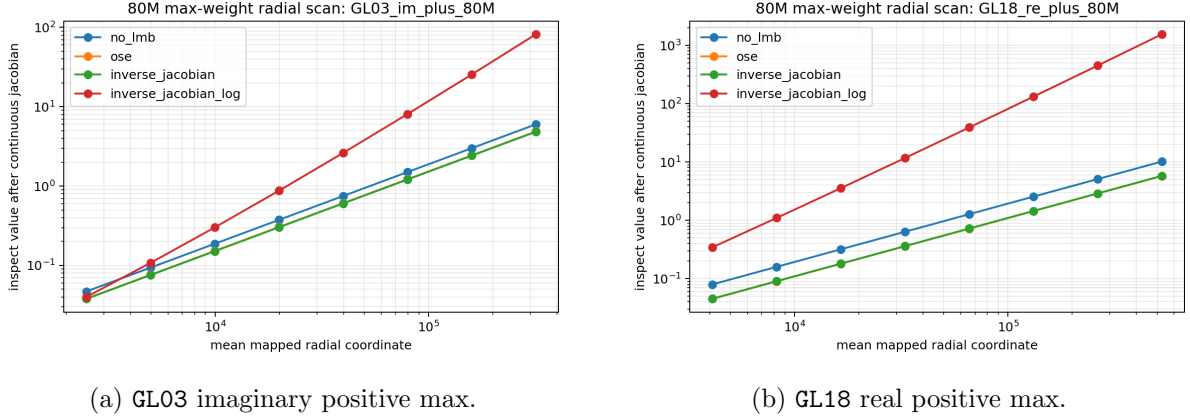


Figure 10: Radial approach scans from two of the 80 M max-weight points. All modes include the continuous sampling Jacobian. The linear inverse-Jacobian LMB prefactor lowers some coefficients, but the positive hard-hard slope remains.

An event-level `inspect` check of GL18 at the recorded max-weight point confirms that all 14 LMB event contributions are present in the summed evaluation. The edge-4 channels are not missing, and their contributions are comparable to the non-edge-4 channels. For example, with inverse-Jacobian weights, channel (8,10) contributes roughly $0.016 + 0.033i$, channel (4,10) contributes $0.036 + 0.036i$, and channel (4,8) contributes $-0.035 - 0.009i$. The 14 channel weights sum to $0.244039 + 0.268363i$, exactly matching the inspected value after the common parameterization Jacobian 1.42839×10^{53} multiplies the pre-Jacobian CFF sum $1.70849 \times 10^{-54} + 1.87877 \times 10^{-54}i$. The parsed event table is stored in `data/current_maxweight_diagnostics/g118_80m_inspect_events.json`. The problem is therefore not that the denominator fails to sum the available LMB maps. The problem is that all available maps still use independent spherical variables for their basis momenta; none introduces the locally natural continuous variables (P_{common}, q) for the small relative momentum q .

This also explains why the edge-4 LMBs do not automatically cure the latest finite max-weight points. In the exactly locked ray, q can remain fixed as the common radius grows. In the actual recorded points the opening angle and radius mismatch are small but not zero, so along the radial scan q grows roughly like $R\theta$. The ratio q/R remains small and still produces a large collinear coefficient, but the edge-4 spherical density sees a hard radial variable and differs from the hard-hard density only by an angular constant. Therefore the inverse-Jacobian denominator can lower coefficients, especially for GL03 and GL18, but it cannot change the observed positive radial power.

The same check was repeated after the additional short control runs, whose largest hard-hard points landed in GL18, GL14, and GL06. `display integrand` confirms that these three diagrams each have 14 generated LMB channels enabled, including all edge-4 combinations. I reran this check directly against the all-graph inverse-Jacobian state; the raw display output and parsed channel list are saved as `data/current_maxweight_diagnostics/all_lmb_display_g106_g114_g118.log` and `.json`. Radial rescans of the new max-weight points again show that inverse-Jacobian LMB weighting changes only coefficients:

point	mode	tail slope	abs. weight at recorded point
GL18 short-run max	no LMB	1.000	0.103
GL18 short-run max	OSE	1.000	0.0269
GL18 short-run max	inverse J	1.000	0.0237
GL14 short-run max	no LMB	1.000	0.0124
GL14 short-run max	OSE	1.000	0.00970
GL14 short-run max	inverse J	1.000	0.00986
GL06 short-run max	no LMB	1.000	0.0205
GL06 short-run max	OSE	1.000	0.0452
GL06 short-run max	inverse J	1.000	0.0424

For these particular short-run points the edge norms computed from the graph momentum signatures are all hard; there is no actual soft edge that an edge-containing LMB channel could use as a small radial variable. Consequently all active LMB inverse Jacobians scale with the same hard power, their normalized ratios approach constants, and the inverse-Jacobian denominator cannot alter the residual hard-hard radial exponent. A common-radius hyperspherical map is a useful diagnostic because it samples the six-dimensional hard radius directly. In practice it did not solve the variance problem: with no LMB multichanneling the 1 M-sample run gave

$$\text{Im} = +6.83 \times 10^{-5} \pm 2.68 \times 10^{-4},$$

and with OSE-weighted summed LMBs it gave

$$\text{Im} = +1.00 \times 10^{-4} \pm 7.36 \times 10^{-5}.$$

The corresponding maximum inspected weights were of order 200 and 69, respectively. A summed inverse-Jacobian hyperspherical run was much slower than the spherical production mode and did not reach a useful checkpoint in the short diagnostic window. Therefore the common-radius map is not currently a practical production fix.

I then repeated the radial scan in selected-channel mode, i.e. with `lmb_channels="monte_carlo"` and explicit `--discrete-dim` selection of each generated LMB channel. This removes any ambiguity about the denominator: if one available chart really resolved the ridge, its selected channel should show a different hard-radius power. It does not. For all three representative hard-edge points, the summed inverse-Jacobian scan and every single selected channel still have slope approximately +1:

point	summed slope	selected-channel slope range	selected abs. range at point	selected abs. range deepest
GL06 short-run max	1.00007	0.99991–1.00007	2.60×10^{-3} – 1.26×10^{-2}	4.16×10^{-2} – 2.02×10^{-1}
GL14 short-run max	0.99976	0.99967–0.99998	3.25×10^{-4} – 2.11×10^{-3}	5.21×10^{-3} – 3.37×10^{-2}
GL18 80 M max	0.99997	0.99996–0.99999	3.14×10^{-2} – 5.04×10^{-2}	5.03×10^{-1} – 8.07×10^{-1}

The vector sum of the selected GL18 channel contributions reproduces the explicit summed-LMB inspect value at both the recorded and deepest scan points, so this is not a bookkeeping mismatch. The output is saved as `data/current_maxweight_diagnostics/hard_edge_ridges_per_lmb_channel_` and `_summary.json`. This is the strongest diagnostic for the current sampler: all LMBs are active, inverse-Jacobian multichanneling is applied, but none of the sampled charts contains the missing repeated-hard-edge coordinate.

I also evaluated the inverse-Jacobian channel densities themselves at three representative ridge points. This diagnostic computes the inverse spherical map density for every generated LMB channel at the same physical point. It is stored as `data/current_maxweight_diagnostics/inverse_jacobian_channel_density_at_ridges.json`. The result again shows that the issue is not a missing channel: the available densities are spread over ordinary hard LMBs rather than enhanced by the repeated internal-edge structure.

point	largest density channel(s)	top fraction	nearest pair
GL06 max	(4, 8), (4, 5)	0.227 each	(6, 9), difference 0
GL14 max	(4, 7), (4, 5), (4, 9), (4, 10)	0.169 each	(6, 8), difference 4.15×10^{-5}
GL18 80 M max	several channels, e.g. (6, 9), (8, 9), (6, 10), (6, 7)	0.080 each	(7, 10), difference 0

For GL06 and GL18 at kinematics A this is especially important: the closest top-edge pair is rank deficient as a coordinate pair, because both edges have the same internal loop signature and differ only by $\vec{P}_0 + \vec{P}_1 = 0$. There is no invertible LMB channel that samples these two edge momenta as independent variables. A production fix for these points therefore needs an edge-cluster or hard-ray channel whose balance density knows that several propagator energies share the same hard three-momentum. A (P_{common}, q) relative-momentum channel is still the right model for genuinely variable small differences, but the exactly locked kinematics-A cases require the repeated-hard-ray limit as well.

The more precise reason is visible from the graph edge signatures. I parsed the generated dot files and evaluated all hard internal edge three-momenta at the recorded short-run and 80 M max-weight points. The nearest edge pairs are not generally the two sampled LMB basis vectors; they are often dependent top edges that differ only by fixed external shifts. For kinematics A, $P_0 + P_1$ has zero spatial part, so some of these pairs are exactly identical as three-momenta:

point	nearest hard edge pair	relative difference	edge representations
GL06 short-run max	e_6, e_9	0	$\vec{k}_1 - \vec{P}_0 - \vec{P}_1, \vec{k}_1$
GL14 short-run max	e_6, e_8	4.15×10^{-5}	$-\vec{k}_0 + \vec{k}_1 - \vec{P}_1 + \vec{P}_2, -\vec{k}_0 + \vec{k}_1 - \vec{P}_1$
GL18 short-run max	e_7, e_{10}	0	$\vec{k}_1 + \vec{P}_2, \vec{k}_1 + \vec{P}_2 - \vec{P}_0 - \vec{P}_1$
GL18 80 M max	e_7, e_{10}	0	$\vec{k}_1 + \vec{P}_2, \vec{k}_1 + \vec{P}_2 - \vec{P}_0 - \vec{P}_1$

This diagnostic is saved in `data/current_maxweight_diagnostics/edge_uv_pair_diagnostics.json` and can be regenerated with `diagnose_edge_uv_pairs.py`. It explains why ordinary all-LMB multichanneling does not change the power: the problematic equal-energy structure is not a missing independent LMB. A relative coordinate built from the two LMB basis vectors also leaves the same R^{+1} tail on the current GL06, GL14, and GL18 rays, because it does not parameterize these dependent edge-pair degeneracies.

9.5 Common-radial and hybrid radial probes

The previous observation suggests a more precise coordinate-level diagnostic. The product spherical map used in the production runs maps each loop radius independently. In a double-UV limit with a fixed ratio between the two loop radii this introduces two large radial derivatives, whereas the physical two-loop scaling has only one common hard radius. I therefore added an opt-in two-loop map

$$(r_0, r_1) = (Ry, R(1 - y)),$$

with ordinary spherical angles for the two loop directions. This `spherical_common_radial` map has the correct common hard-radius Jacobian for the double-UV diagonal.

Rescanning the 80 M max-weight points with the same physical momenta transformed into this coordinate system confirms the diagnosis: the old R^{+1} tails disappear for GL01, GL03, and GL18, and are strongly reduced for GL17.

point	old inverse- J slope	common-radial summed slope	selected-channel slope range
GL01 80 M max	1.0002	0.00039	0.00011–0.00072
GL03 80 M max	1.0000	−0.00002	−0.00002–0.00001
GL17 80 M max	1.0980	0.101	−0.013–0.637
GL18 80 M max	1.0000	−0.00001	−0.00002–0.00001

The raw scan output is saved as `data/current_maxweight_diagnostics/common_radial_ridge_scan.json` and `_summary.json`.

The pure common-radial map is not a production map by itself. It handles the double-hard diagonal, but it undersamples single-hard-loop boundaries $y \rightarrow 0$ and $y \rightarrow 1$. A 1 M-sample all-graph run with summed inverse-Jacobian LMBs found new maxima at $y \simeq 4 \times 10^{-4}$ and gave

$$\text{Im} = -1.62 \times 10^{-6} \pm 1.99 \times 10^{-6}.$$

This is worse than the product-spherical production estimator.

I then implemented an experimental hybrid map `spherical_product_common_radial`. Half of the first integration coordinate selects the ordinary product-spherical radial map, and the other half selects the common-radial map. The sampled branch is tagged on the momentum sample; for inverse-Jacobian LMB multichanneling the numerator uses the inverse density of the sampled branch and the denominator sums both product and common-radial densities over all LMBs. This keeps the product radial sectors for single-hard-loop limits while adding a true common-radius sector for double-hard limits.

At 1 M samples, the hybrid map removed the visible hard-radial maxima: all max-weight points were bulk-like and below 0.15, with negligible instability. However, a 10 M-sample run found a new GL00 common-branch point,

$$x = (0.9999999761, 0.68264, 0.21318, 0.82083, 0.22210, 0.90592),$$

with maximum imaginary weight about 1.07×10^3 . The resulting integral estimate was dominated by this point,

$$\text{Im} = +1.05 \times 10^{-4} \pm 1.07 \times 10^{-4},$$

and is therefore unusable.

`display integrand` confirms that GL00 has all 11 generated LMB channels enabled:

$$(5, 10), (4, 10), (5, 9), (4, 9), (5, 8), (4, 8), (5, 7), (4, 7), (5, 6), (4, 6), (4, 5).$$

A direct radial rescan of the GL00 hybrid max with all 11 selected channels gives slope +1.000 in every channel:

mode	slope	abs. at recorded point	deepest selected abs. range
summed LMBs	1.0000	42.6	—
selected LMB channels	1.0000–1.0000	2.03–7.06	32.5–84.2

This is qualitatively different from the earlier parameterization artifact. Because the common-radial Jacobian has the correct double-UV measure, a remaining event-weight slope R^{+1} corresponds to a physical two-loop UV behavior close to degree zero along this GL00 ray. I attempted to confirm this with `profile ultra-violet` using the same fixed ray and max scaling 10^8 , but the graph-unrestricted profile run reached about 91 GB RSS and was stopped before the 100 GB safety threshold. The safer direct inspect scan is therefore the current evidence. The data are stored as `all_lmb_display_gl00.json`, `hybrid_gl00_max_radial_scan.json`, and `hybrid_gl00_max_radial_scan_summary.json`.

The conclusion from these probes is narrower but important. The original GL01/GL03/GL18 ridges are indeed caused by the product-radial parameterization over-resolving a common hard radius. A hybrid continuous channel can remove those ridges. The full amplitude is nevertheless still blocked by a GL00 two-loop UV ray that appears to survive even in the common-radius measure. That points back to the local UV subtraction or the generated integrated-CT structure for GL00, rather than to LMB channel selection.

Two additional controls exclude simple alternatives. First, I regenerated GL18 alone with tropical subgraph tables enabled and integrated it with `coordinate_system="tropical"`. The 100 k-sample run gave

$$\text{Im} = -6.75 \times 10^{-6} \pm 1.81 \times 10^{-5}$$

at about 2.64 ms per sample per core, with maximum-weight impact 1.52. This is much worse than the spherical inverse-Jacobian LMB estimator scaled to the same graph/sample count, so the current tropical sampler does not capture this repeated-hard-ray feature. The card, log, and result are stored as `aa_aa_2l_gl18_tropical_probe.toml`, `data/current_maxweight_diagnostics/gl18_tropical_probe.toml`, and `.../gl18_tropical_probe_100k_result.json`. I also attempted a short fresh-generation probe with nondefault tropical `target_omega` values. That did not produce an integration result: the fresh GL18 state failed during the integrated-UV/vakint tensor-reduction stage before sampling began. The log is saved as `data/current_maxweight_diagnostics/gl18_tropical_omega_2p0_100k.log`. This is a generation-side issue, not evidence that the tuned tropical sampler improves or worsens the ridge, so I did not use it in the statistical comparison.

Second, I tested whether the existing OSE balance weight could be pushed into the hard tail by choosing $\alpha = -1$, so that the OSE factor grows with the hard energies rather than suppressing them. With all graphs, summed orientations, summed LMBs, kinematics A, and helicity $+-+-$, the 1 M-sample run gave

$$\text{Im} = +2.94 \times 10^{-6} \pm 3.38 \times 10^{-5}.$$

The largest signed weights rose to about 64 in the real part and 16 in the imaginary part. This overcorrects badly and moves variance into other regions. The result is saved as `data/current_maxweight_diagnostics/ose_alpha_minus1_1m_result.json`. A useful fix must therefore be more local than a global sign flip of the OSE exponent.

9.6 Radial power-map and precision controls

A follow-up test asked whether the ridge could be handled by changing only the global radial density. For the 80 M GL03 max-weight ray I transformed the same physical momenta to the power map

$$r = E_{\text{cm}} b \left(\frac{x}{1-x} \right)^p$$

and rescanned the full all-graph summed inspected weight. With $b = 1$, the large-radius slopes were:

power p	fitted slope on the GL03 ray
1.25	+0.600
1.50	+0.332
2.00	−0.007 to −0.04 over the scanned range
3.00	about −0.36
4.00	about −0.56

Thus $p = 2$ is the first simple radial map that neutralizes the observed linear hard-radius growth on this fixed ray. This confirms that the ridge is partly a mismatch between the hard-radius tail and the sampling density.

However, the full integration test is less favorable. The table below shows matched 1 M-sample graph-Monte-Carlo, summed-LMB runs with kinematics A and helicity $+-+-$:

sampling	Im value	Im error	f128	Arb	nan/unstable
linear, inverse- J LMB	-1.41×10^{-6}	2.25×10^{-6}	0.0018%	0	0
relative spherical, inverse- J LMB, 1.45 M samples	-1.35×10^{-6}	1.21×10^{-6}	0.00145%	0	0
power $p = 1.5$, inverse- J LMB	$+1.28 \times 10^{-6}$	2.26×10^{-6}	0.0716%	0	$3.0 \times 10^{-4}\%$
power $p = 1.75$, inverse- J LMB	-0.46×10^{-6}	2.40×10^{-6}	0.1799%	0	$2.4 \times 10^{-3}\%$
power $p = 2$, quad for high norms	-15.7×10^{-6}	12.4×10^{-6}	0.4109%	0	$7.8 \times 10^{-3}\%$

The relative-spherical control used all generated LMB channels and the inverse-Jacobian balance denominator, and its maximum signed weights stayed at $\mathcal{O}(10^{-1})$. Nevertheless the total imaginary relative error was still about 90% at 1.45 M samples, so the control did not change the

full-amplitude cost scaling. This is consistent with the edge-pair diagnostic above: the hard-hard equal-momentum pairs are dependent internal edges, not generally the two LMB basis vectors used by the relative-spherical map. The checkpoint is stored as `data/additional_sampling_controls/relative_integration_result_iter_0010.json`.

The $p = 1.5$ and $p = 1.75$ maps do not improve the absolute error. The $p = 2$ map overexposes very deep UV samples. Several apparent $\mathcal{O}(1)$ to $\mathcal{O}(10^4)$ max weights in these runs are double- or quad-precision cancellation artifacts: direct arbitrary-precision `inspect` at the same points gives ordinary weights of order 10^{-5} or smaller. For example, the GL00 $p = 2$ point printed $+1.16 \times 10^4 i$ in the integration max table but evaluates to $-8.2 \times 10^{-6} - 9.1 \times 10^{-6} i$ in arbitrary precision. A GL08 $p = 2$ point printed a weight around $-12i$ but evaluates to $-3.1 \times 10^{-7} - 1.1 \times 10^{-6} i$ in arbitrary precision.

Forcing high loop-momentum norms directly to arbitrary precision removes those fake max weights. In a 100 k-sample $p = 2$ control with $\sum |\vec{k}_i| > 10^6 E_{\text{cm}}$ escalated to the final Arb level, the precision mix was 99.519% f64, 0.295% f128, 0.186% Arb, and 0 nan/unstable samples. The max weights then returned to $\mathcal{O}(0.1)$. The statistical error, however, remained comparable to the linear baseline when scaled to 1 M samples. This means the global radial map can regularize selected approach plots, but it is not yet an efficient production sampler for the full amplitude.

The practical conclusion from this test is that a global hard-radius density is too blunt. The required channel should target the asymptotic OSE cluster, the repeated hard ray, or a genuinely small hard-hard relative variable locally, while leaving the rest of the UV domain to the ordinary spherical/LMB channels. Such a channel must also trigger arbitrary precision in the very-deep-UV cancellation region, or use a more reliable stability criterion there.

The proxy channel-partition scan gives a concrete target for such a fix. In the standalone proxy, the raw bad hard-hard channel grows as R^{+1} when the relative momentum q is held fixed. If the balance denominator also contains a true (P_{common}, q) channel, the bad direct-channel contribution instead scales as R^{-3} with the inverse-Jacobian density for the linear map, and as R^{-2} with an OSE $\alpha = 3$ proxy. This is the behavior the current GL00 estimator is missing in the variable- q case: the implemented inverse-Jacobian denominator is correct for the existing LMB maps, but the existing map set does not include the dependent internal-edge relative channel. For the exactly locked kinematics-A pairs, where q is fixed by the external spatial shift, the analogous missing object is a repeated-hard-ray edge-cluster density rather than an invertible two-edge LMB.

10 Implications

The current evidence points to a high-variance, locally integrable relative-momentum singularity rather than a wrong integrated UV counterterm or a missing local UV cancellation. The practical options suggested by the toy model are:

1. add a specialized continuous channel or remapping for hard-hard relative momentum ridges and repeated-hard-ray edge clusters. For variable relative momentum, a concrete implementation target is a map that samples variables close to (P_{common}, q) , for example $k = P_{\text{common}} + q/2 + \Delta$ and $l = P_{\text{common}} - q/2 + \Delta'$, with the external shifts chosen from the small internal edge under consideration. For the exactly locked kinematics-A cases, where the two top-edge three-momenta are the same function of one loop momentum, the channel should instead encode the repeated-edge hard-ray multiplicity. In both cases the channel density must enter the same balance denominator as the LMB channels through its actual inverse Jacobian or another normalized positive channel weight assigned to a sampled channel;
2. avoid using a global OSE exponent scan as the primary cure. The $\alpha = -1$ control shows that a hard-tail sign flip overcorrects and produces much larger max weights. Any useful

OSE-based variant has to be local to the repeated hard-edge cluster;

3. use explicit summed LMB channels as a diagnostic, while recognizing that this reduces discrete-channel variance but does not by itself remap the continuous relative-ridge variables.